Name: M. Sivateja

Hall Ticket: 2303A52145

Batch: 41

Task 1: AI-Assisted Bug Detection

Prompt:

Analyze the following Python function for calculating factorial.
Identify any logical errors, explain the issue, and provide a corrected version.
Also consider edge cases like 0 and negative numbers.

Code:

```
def factorial(n):
    result = 1
    for i in range(1, n):
        result = result * i
    return result

print(factorial(5))
```

output:

24

Loop runs 1 → n-1 Missing n → off-by-one error

Code 2:

```
def factorial(n):
    if n < 0:
        raise ValueError("Negative not allowed")
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print(factorial(5))
```

n = 0 → 1 negative → error

Explanation:

The function contains an off-by-one error because the loop runs only up to n-1 instead of n. This results in incorrect factorial values. The corrected version includes n in the loop and handles edge cases like zero and negative inputs properly

Task 2: Improving Readability & Documentation

Review the following Python function for readability and maintainability.
Improve the function by renaming variables, adding a proper docstring,
handling division by zero, and validating inputs.

Code:

```python
def calculate(a: float, b: float, operation: str) -> float:
    """
    Performs arithmetic operations.

    Parameters:
        a (float): First number
        b (float): Second number
        operation (str): 'add', 'sub', 'mul', 'div'

    Returns:
        float: Result

    Raises:
        ValueError: Invalid operation
        ZeroDivisionError: Division by zero

    Example:
        calculate(10, 5, 'add') -> 15
    """
    if not isinstance(operation, str):
        raise TypeError("Operation must be string")

    if operation == "add":
        return a + b
    elif operation == "sub":
        return a - b
    elif operation == "mul":
        return a * b
    elif operation == "div":
        if b == 0:
            raise ZeroDivisionError("Cannot divide by zero")
        return a / b
    else:
        raise ValueError("Invalid operation")
```

Testing:

```python
print(calculate(10, 5, "add"))
try:
    print(calculate(10, 0, "div"))
except ZeroDivisionError as e:
    print(f"Error: {e}")
```

Explanation: The original function lacks descriptive naming and proper documentation, making it difficult to understand. The improved version uses meaningful names, includes a docstring, and adds error handling for invalid inputs and division by zero. This enhances readability, usability, and robustness**.**

Task 3: Enforcing Coding Standards

Prompt:

Check the following Python function for PEP8 compliance.
List all violations and refactor the code to follow proper naming conventions, indentation, and readability while keeping functionality intact.

Code:

```
def check_prime(n: int) -> bool:
    if n < 2:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Test

```
print(check_prime(7))  # True
print(check_prime(10)) # False
```

Output:

True

False

Explanation:

The function violates PEP8 standards by using improper naming conventions and inconsistent formatting. The refactored version uses lowercase function names and better structure for readability. Following coding standards improves code consistency and maintainability in team projects.

Task 4: AI as a Code Reviewer in Real Projects

Prompt:

Act as a code reviewer and analyze the following function.
Suggest improvements in naming, input validation, and reusability.
Add type hints and make the function more flexible.

Code:

```
from typing import List, Union

def multiply_even_numbers(
    numbers: List[Union[int, float]],
    multiplier: int = 2
) -> List[Union[int, float]]:

    if not isinstance(numbers, list):
```

```
        raise TypeError("Input must be a list")

    result = []
    for num in numbers:
        if isinstance(num, (int, float)) and num % 2 == 0:
            result.append(num * multiplier)

    return result
```

Test:

```
print(multiply_even_numbers([1,2,3,4]))
```

output:

```
•••  [4, 8]
```

Explanation:

The original function lacks clear naming and input validation, reducing its usability. The improved version introduces descriptive naming, type hints, and error handling. It also enhances flexibility by allowing configurable operations, making the function more reusable.

Task 5: AI-Assisted Performance Optimization

Prompt:

Analyze the performance of the following function.
Explain its time complexity and suggest optimizations.
Provide a more efficient implementation if possible.

Code:

Original:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```

Optimized:

```
def sum_of_squares_optimized(numbers):
    return sum(x * x for x in numbers)
```

Test Performance:

```
import time

nums = list(range(1000000))

start = time.time()
sum_of_squares(nums)
print("Old:", time.time() - start)

start = time.time()
sum_of_squares_optimized(nums)
print("New:", time.time() - start)
```

Complexity:

- Both: **O(n)**
- Optimized → faster due to internal implementation

output:

```
•••  Old: 0.08806467056274414
     New: 0.07051682472229004
```

Explanation:

The function has a time complexity of O(n) as it iterates through all elements. The optimized version uses Python's built-in sum function with a generator expression, improving execution speed. This approach enhances performance while maintaining readability.